

Dokumentacja techniczna projektu

Projekt: Automatyczne parkowanie w Unity 3D

Wersja dokumentacji: 2026-06-15

Charakter projektu: deterministyczna symulacja parkowania, bez machine learningu

1. Cel projektu

Celem projektu jest pokazanie prostego systemu automatycznego parkowania samochodu w Unity. Program ma:

- uruchamiać się w Unity jako zwykła scena 3D,
- mieć trzy scenariusze testowe,
- używać czujników opartych o Raycast i CheckBox,
- sterować autem przez maszyny stanów FSM,
- reagować na przeszkodę dynamiczną,
- pokazać wynik w czytelny sposób na ekranie,
- być możliwy do wytłumaczenia bez znajomości ML-Agents i bez trenowania modelu.

Projekt nie jest symulatorem profesjonalnej fizyki pojazdów. To wersja zaliczeniowa: stabilna, powtarzalna i prosta do omówienia. Najważniejsze jest pokazanie algorytmu, podziału na stany, sensorów i reakcji na sytuacje na parkingu.

2. Główna idea działania

Auto nie uczy się samo. Działa według deterministycznego scenariusza:

1. Auto jedzie powoli wzdłuż drogi.
2. Sensory mierzą odległości od przeszkód.
3. Program publikuje informacje o znalezionym miejscu parkingowym.
4. Maszyna stanów przechodzi przez kolejne etapy parkowania.
5. W mapach demonstracyjnych finalny manewr jest wykonywany po wyznaczonych punktach, jak po szynie.
6. Po dojechaniu do ostatniego punktu auto zatrzymuje się w stanie Parked.

Finalna wersja celowo używa trybu "po szynach" dla manewru parkowania. Powód: w Unity prosta, niekonfigurowana fizyka pojazdu potrafi dawać niestabilne wyniki, a projekt ma działać przewidywalnie podczas prezentacji.

3. Struktura folderów

Po zainstalowaniu paczki w projekcie Unity ważne foldery są takie:

[blok kodu / diagram w pliku Markdown]

Folder Documentation znajduje sie w paczce poza Assets, zeby nie mieszac dokumentow z logika Unity.

4. Najwazniejsze klasy

Plik Rola

--- ---

ParkingTypes.cs Definicje typow danych: stany FSM, typ parkowania, odczyty sensorow, kandydat miejsca.

ParkingCarController.cs Sterowanie pojazdem: predkosc, skret, hamowanie, ruch kinematyczny i ruch po punktach.

ParkingSensors.cs Czujniki Raycast, wykrywanie luk i walidacja wolnej przestrzeni.

ParkingStateMachine.cs Glowna logika FSM: skanowanie, walidacja, manewr, stop awaryjny, zaparkowanie.

ParkingScenario.cs Dane scenariusza: punkt akceptacji miejsca, lista punktow manewru, opis wykrytej luki.

MovingObstacle.cs Ruch czerwonego auta i wymuszenie zatrzymania niebieskiego auta.

ParkingDemoBuilder.cs Generator materialow, prefabu auta, UI i trzech scen testowych.

VehicleVisualRig.cs Wizualny obrot kol i skret przednich kol.

DebugHud.cs Tekst debug w lewym gornym rogu.

MapManager.cs Ladowanie scen i restart.

ImmediateMapUi.cs Zapasowe przyciski OnGUI do zmiany map.

CameraFollow.cs Kamera sledzaca auto.

5. Dane publiczne i struktury

ParkingState

Enum opisuje aktualny stan automatu:

[blok kodu / diagram w pliku Markdown]

Znaczenie stanow:

- Scan - auto jedzie i szuka miejsca.
- ValidateSpot - program potwierdza znalezione miejsce.
- Positioning - auto ustawia sie do manewru.
- ReverseTurn - pierwszy skret przy wjezdzie w miejsce.
- CounterTurn - kontra, czyli drugi etap ustawiania auta.
- Straighten - prostowanie auta w miejscu.
- Parked - koniec.
- EmergencyStop - zatrzymanie awaryjne, np. przy przeszkodzie dynamicznej.

ParkingType

[blok kodu / diagram w pliku Markdown]

Perpendicular oznacza parkowanie prostopadle, Parallel oznacza parkowanie rownolegle.

SensorReadings

Struktura przechowuje odleglosci z sensorow:

[blok kodu / diagram w pliku Markdown]

Przyklad: rightFront to odleglosc po prawej stronie z przodu pojazdu, front to odleglosc przed pojazdem.

ParkingSpotCandidate

Opisuje kandydat na miejsce parkingowe:

[blok kodu / diagram w pliku Markdown]

W scenach demonstracyjnych kandydat jest czesciowo publikowany z ParkingScenario, zeby prezentacja byla powtarzalna.

6. Maszyna stanow FSM

FSM znajduje sie w ParkingStateMachine.cs. To najwazniejszy skrypt projektu.

Schemat:

[blok kodu / diagram w pliku Markdown]

Awake

W Awake pobierane sa komponenty:

- ParkingCarController,
- ParkingSensors,
- ParkingManeuverPlanner,
- opcjonalnie ParkingScenario.

Jezeli obiekt ma ParkingScenario, to:

[blok kodu / diagram w pliku Markdown]

Oznacza to, że ruch auta będzie sterowany przez punkty scenariusza, a nie przez swobodną fizykę.

FixedUpdate

FixedUpdate działa w stałym kroku fizycznym Unity. W projekcie jest to dobre miejsce na logikę ruchu auta.

Najpierw aktualizowany jest czas stanu:

[blok kodu / diagram w pliku Markdown]

Potem sprawdzany jest stop awaryjny:

- raycastEmergency - przeszkoda z sensorów,
- scriptedEmergency - przeszkoda dynamiczna wymuszona przez MovingObstacle.

Jeśli auto jest w trakcie manewru i pojawia się przeszkoda, FSM przechodzi do EmergencyStop.

UpdateScenarioScan

W scenach demonstracyjnych auto nie czeka na przypadkowy wynik sensorów, tylko dojeżdża do punktu scanAcceptPoint.

Kiedy dojedzie:

1. budowany jest ParkingSpotCandidate,
2. kandydat jest wyświetlany w HUD,
3. FSM przechodzi do ValidateSpot.

UpdateScenarioValidation

Auto zatrzymuje się na krótką pauzę:

[blok kodu / diagram w pliku Markdown]

Potem rozpoczyna pierwszy krok scenariusza.

UpdateScenarioManeuverState

To metoda, która wykonuje punkty manewru:

1. Pobiera aktualny ParkingScenarioStep.
2. Jeśli krok ma smoothArc = true, używa UpdateSmoothScenarioStep.
3. Jeśli nie, używa prostego MoveToPose.
4. Po zakończeniu kroku przechodzi do kolejnego.

Nazwa smoothArc została historycznie, ale w finalnej wersji ruch jest celowo zrobiony "po szynach": prosta interpolacja pozycji między punktami i łagodna interpolacja kąta.

UpdateSmoothScenarioStep

Finalna wersja działa tak:

[blok kodu / diagram w pliku Markdown]

Na starcie kroku zapamiętujemy:

- pozycje startowa,
- kat startowy,
- roznice kata do punktu docelowego,
- dlugosc odcinka.

Potem w kazdym kroku:

[blok kodu / diagram w pliku Markdown]

To znaczy:

- stepProgress rośnie od 0 do 1,
- Vector3.Lerp przesuwaa auto po prostej szynie,
- Mathf.SmoothStep łagodzi start i koniec obrotu,
- Mathf.DeltaAngle gwarantuje najkrotszy obrot, bez 360 stopni.

Krok konczy sie, gdy:

[blok kodu / diagram w pliku Markdown]

Nie ma już dokrecania w miejscu. To ważne, bo wcześniejsze wersje potrafiły stac i obracać samochód.

7. Sterowanie autem

Sterowanie znajduje sie w ParkingCarController.cs.

Dwa tryby pracy

1. Tryb kinematyczny:

- uzywany, gdy nie ma ParkingScenario,
- auto reaguje na SetControls(throttle, steering, brake),
- obrot liczony jest uproszczonym modelem rowerowym.

2. Tryb po punktach:

- uzywany w scenach demonstracyjnych,
- UseExternalPoseControl = true,
- FSM ustawia pozycje przez SetPoseAlongPath.

SetControls

[blok kodu / diagram w pliku Markdown]

Parametry:

- throttle od -1 do 1, ujemny oznacza cofanie,
- steering od -1 do 1, kierunek skretu,
- brake od 0 do 1.

Kinematyka w FixedUpdate

Predkosc nie zmienia sie natychmiast. Uzywany jest Mathf.MoveTowards, czyli plynne dojscie do predkosci docelowej.

Skret tez nie zmienia sie od razu:

[blok kodu / diagram w pliku Markdown]

Obrot auta w trybie kinematycznym:

[blok kodu / diagram w pliku Markdown]

To uproszczony model rowerowy. Zaklada, ze pojazd ma jeden wirtualny przod i jeden wirtualny tyl.

SetPoseAlongPath

Ta metoda jest kluczowa dla finalnego trybu:

[blok kodu / diagram w pliku Markdown]

Ustawia:

- pozycje auta,
- rotacje tylko po osi Y,
- aktualna predkosc do HUD i kol,
- odpowiedz skretu dla wizualnych kol.

Funkcja FlattenYaw zeruje niepotrzebne obroty w osiach X i Z, zeby auto nie przechylalo sie ani nie obracalo przypadkowo.

8. Sensory

Sensory sa w ParkingSensors.cs.

Odczyty Raycast

Metoda GetReadings wykonuje kilka promieni:

- przod,
- tyl,
- prawa strona: przod, srodek, tyl,
- lewa strona: przod, srodek, tyl.

Promienie sa rysowane w Scene View:

- czerwony oznacza trafienie w przeszkode,
- zielony oznacza brak przeszkody.

Najwazniejsza metoda pomocnicza:

[blok kodu / diagram w pliku Markdown]

Zwraca odleglosc do przeszkody albo maksymalny zasieg.

Wykrywanie luki

TryFindSpot sprawdza, czy po prawej stronie jest wolna przestrzen:

[blok kodu / diagram w pliku Markdown]

Jesli wolna przestrzen trwa odpowiednio dlugo, powstaje kandydat na miejsce.

Walidacja objetosci

Sama odleglosc z promieni nie wystarczy. Dlatego jest dodatkowy test:

[blok kodu / diagram w pliku Markdown]

CheckBox sprawdza, czy w przewidywanym miejscu parkingowym nie ma obiektu. To zabezpiecza przed falszywa luka.

9. Scenariusze parkowania

Scenariusz jest w ParkingScenario.cs.

Najwazniejsze pola:

- parkingType - prostopadle albo rownolegle,
- scanAcceptPoint - gdzie auto uznaje miejsce za znalezione,
- scanSpeed - predkosc podczas dojazdu,
- validatePause - pauza po znalezieniu miejsca,
- spotStart, spotEnd, spotWidth - opis miejsca do HUD,
- steps - lista punktow manewru.

Pojedynczy krok:

[blok kodu / diagram w pliku Markdown]

W finalnej wersji najwazniejsze sa:

- state,
- target,
- reverse,
- speed,
- timeout.

curveHandle zostal jako pole kompatybilne z poprzednimi wersjami, ale aktualny ruch po szynach nie korzysta z krzywych Beziera.

10. Generator scen

ParkingDemoBuilder.cs jest skryptem edytorowym. Nie działa w buildzie jako logika gry, tylko generuje zasoby w Unity.

Menu:

[blok kodu / diagram w pliku Markdown]

Po kliknięciu wykonywane jest:

[blok kodu / diagram w pliku Markdown]

Ta metoda:

1. tworzy foldery,
2. tworzy materiały,
3. tworzy prefaby,
4. generuje menu główne,
5. generuje mapę 1,
6. generuje mapę 2,
7. generuje mapę 3,
8. ustawia sceny w Build Settings.

Materiały

CreateMaterials tworzy materiały:

- asfalt,
- beton,
- linie parkingowe,
- niebieski samochód,
- szare samochody,
- czerwony samochód dynamiczny,
- ściany,
- opony,
- szyby.

Model auta

AddVehicleVisual tworzy auto z prymitywów Unity:

- Body - główna bryła auta,
- Cabin - kabina/szyby,
- YellowFrontMarker albo FrontLights - oznaczenie przodu,

- 4 walce jako kola.

To nie jest realistyczny model 3D, ale pozwala czytelnie pokazac przod, tyl i skret.

11. Mapa 1

Nazwa: Map_01_Perpendicular

Zalozenie:

- parking prostopadly,
- wiekszosc miejsc zajeta,
- jedna luka jest za waska,
- druga luka jest miejscem docelowym.

Auto startuje:

[blok kodu / diagram w pliku Markdown]

Miejsca i auta sa po prawej stronie. Auto ma dojechac do punktu skanowania, zignorowac zla luke i zaparkowac w miejscu docelowym.

Tor manewru jest zapisany w ConfigurePerpendicularScenario.

12. Mapa 2

Nazwa: Map_02_Parallel

Zalozenie:

- waska ulica,
- parkowanie rownolegle,
- auto ma wykonac cofanie i kontre,
- finalnie ma stac rownolegle do drogi.

Tor jest zapisany w ConfigureParallelScenario.

13. Mapa 3

Nazwa: Map_03_Dynamic

Zalozenie:

- scena podobna do mapy 1,
- miejsca parkingowe sa po lewej stronie,
- czerwone auto jedzie z naprzeciwka,

- niebieskie auto musi zaczekać,
- po przejechaniu czerwonego auta niebieskie parkuje po lewej.

Najważniejsze punkty:

[blok kodu / diagram w pliku Markdown]

Finalne miejsce jest między liniami:

- $z = -10.1$,
- $z = -6.7$.

Finalny punkt niebieskiego auta:

[blok kodu / diagram w pliku Markdown]

To środek miejsca parkingowego.

14. Przeszkoda dynamiczna

MovingObstacle.cs przesuwa czerwone auto między pointA i pointB.

W mapie 3:

- czerwone auto startuje z przodu,
- rusza, gdy niebieskie jest w stanie Scan,
- wymusza EmergencyStop przez RequestEmergencyStop,
- po dojechaniu do końca może się zatrzymać.

Klucz:

[blok kodu / diagram w pliku Markdown]

To nie jest fizyczne zderzenie. To celowe wymuszenie reakcji FSM.

15. HUD i debug

DebugHud.cs pokazuje:

- aktualny stan FSM,
- prędkość auta,
- kąt skrętu,

- odczyty sensorow,
- czy trwa sledzenie luki,
- ostatniego kandydata miejsca parkingowego.

To pomaga podczas prezentacji, bo widac, ze system faktycznie przechodzi przez stany.

16. Kamera

CameraFollow.cs sledzi auto:

[blok kodu / diagram w pliku Markdown]

Kamera nie jest elementem algorytmu, tylko ulatwia obserwacje.

17. UI

Projekt ma dwa typy UI:

1. Canvas z przyciskami tworzony przez ParkingDemoBuilder.
2. Awaryjne UI OnGUI z ImmediateMapUi.

OnGUI jest malo eleganckie, ale praktyczne: nawet jesli Canvas nie zadzialalby poprawnie, dalej mozna zmieniac mapy.

18. Dlaczego nie ML-Agents

Projekt nie uzywa ML-Agents, bo:

- uczenie modelu zajmuje duzo czasu,
- wynik uczenia nie jest deterministyczny,
- trudniej wyjasnic decyzje auta,
- na projekt zaliczeniowy wystarcza FSM + sensory + scenariusze.

To jest zaleta w dokumentacji: prowadzacy moze zapytac "dlaczego bez ML?", a odpowiedz brzmi: wymagany byl algorytm automatycznego parkowania, niekoniecznie uczony model; zastosowano deterministyczny automat stanow i sensory.

19. Ograniczenia projektu

Najwazniejsze ograniczenia:

- auto nie ma pelnego modelu zawieszenia,
- finalny manewr nie jest obliczany dynamicznie z geometrii miejsca, tylko zapisany punktami,
- trasy sa dopasowane do przygotowanych scen,
- kolizje nie sa glownym mechanizmem sterowania,

- projekt pokazuje zasady działania, a nie gotowy system do prawdziwego auta.

Te ograniczenia są akceptowalne, jeśli są uczciwie opisane.

20. Co mówić przy prezentacji

Krótkie wytłumaczenie:

"Projekt pokazuje automatyczne parkowanie zrealizowane deterministycznie. Auto ma sensory Raycast, które mierzą odległości od przeszkód, a logika jest oparta o maszyny stanów. Po znalezieniu miejsca auto wykonuje manewr według zaprogramowanej trajektorii punktowej. W trzeciej scenie pojawia się auto z naprzeciwka, które wymusza stan EmergencyStop. Rozwiązanie nie używa ML, bo celem była powtarzalna demonstracja algorytmu i reakcji na przeszkodę."